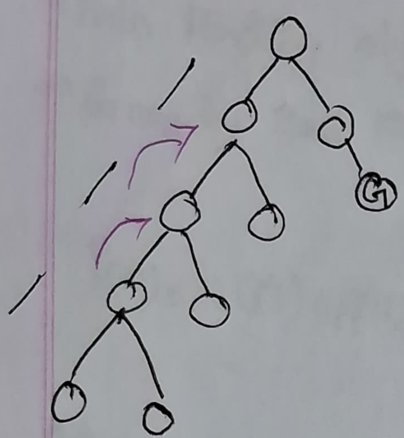
Local Search



$\boxed{\text{backtrack}}$ is a crucial factor in tree/graph.

$\boxed{\text{path cost}}$ matter করে।

save থাকে

fringe এর কোন path হয়ে কোন path এ ~~[illegible]~~ যায়-

কোন node visit হয়।

→ heuristic দিয়ে agentic AI related graph solve করতে time লাগে,

⇒ So AI তে (not concerned with path) এমন way আছে graph solve এর,

Local Search

এখানে matter করে solution itself.

→ Not concerned with perfect sol

→ Not needed for memory usage

→ perfect sol না হলেও হবে

→ close enough ans পেলেও হবে যদি back tracking এর necessity কমে যায়,

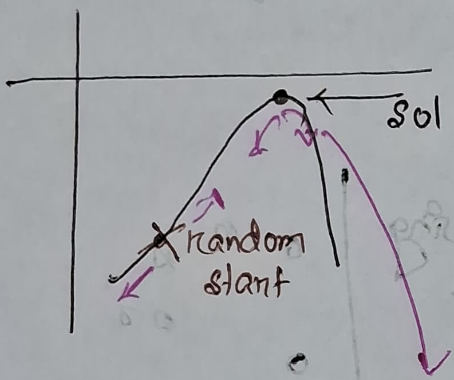
→ node এর info fringe এ save করা লাগবেনা

→ memory save হবে।

Local search advantages:

1. less memory
2. less time
3. applicable for large size graph

Hill climb search



might be 2D/3D/4D

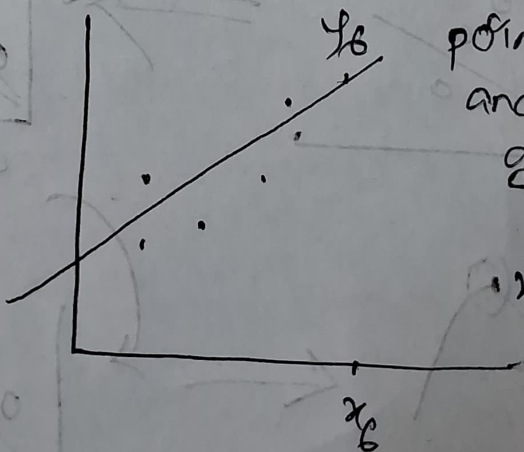
blind folded থাকলে যে দেখবে কোন direction এ গেলে upward এ যাচ্ছে।

এমন একটা point left or right দেখবে সাথে descend হচ্ছে। so ওটাই goal.

x_1	y_1
x_2	y_2
x_3	y_3
x_4	y_4
x_5	y_5
x_6	~~y_6~~ ?

$y = mx$

Linear regression



point plot করি and line টা generalize করে দেই।

- x_6 এর against এ y_6 আসবে।

know

unknown will know

$y = mx + b$

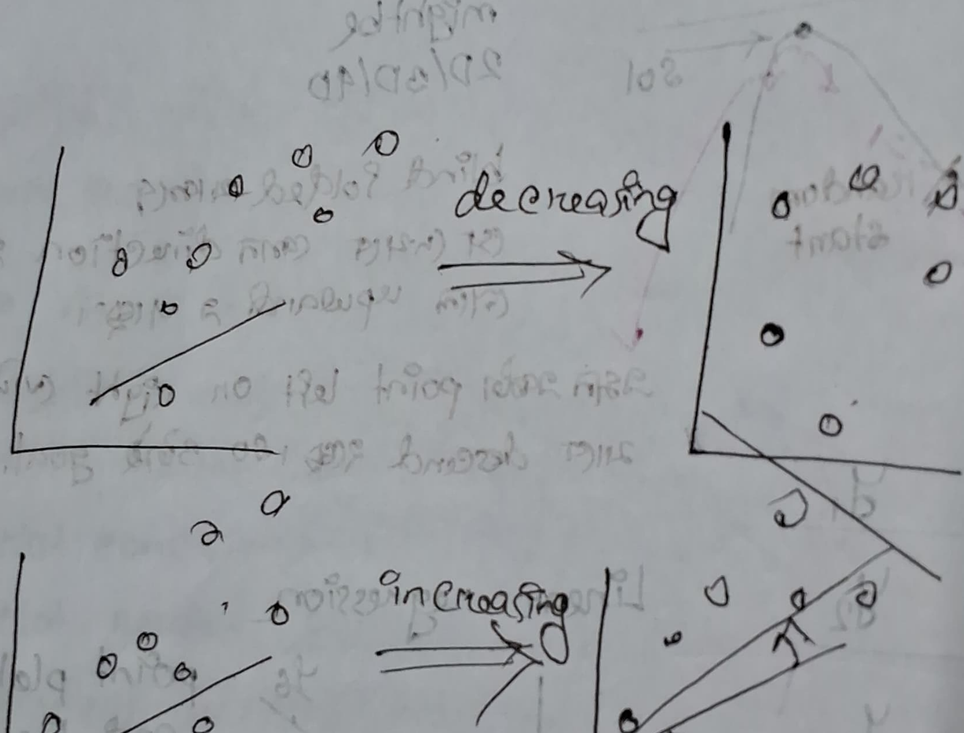
slope

intercept

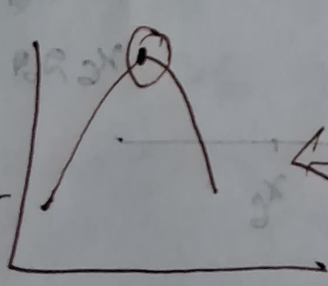
$m \rightarrow \boxed{-\infty \le m \le \infty}$

infinite search space

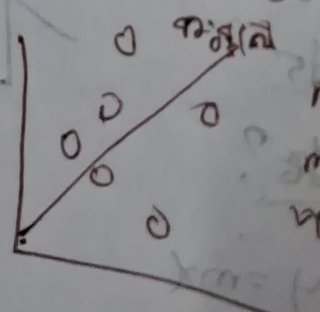
→ only solution matter



যখন Further increment করলে



Further ← increment করলে error আবার বেড়ে যায়।

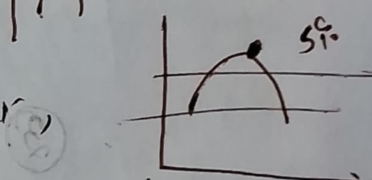


m এর top most point পাবো, error minimum হবে।

goal aim reached বোঝা যাবে তখন, যখন perfect timing এর পাওয়া গেলে m এর value কমালেও error বাড়বে, বাড়ালেও error বাড়বে.

advantage
- → scalable
- → path matter করেনা-
- → final value/ end result নিয়ে only concerned
- → m এর জন্য ~~spa~~ state space infinit হচ্ছে, এখানে memory লাগছেনা
- → কিছুই track রাখা লাগছেনা simulation এ just m বাড়ে।

→ single top



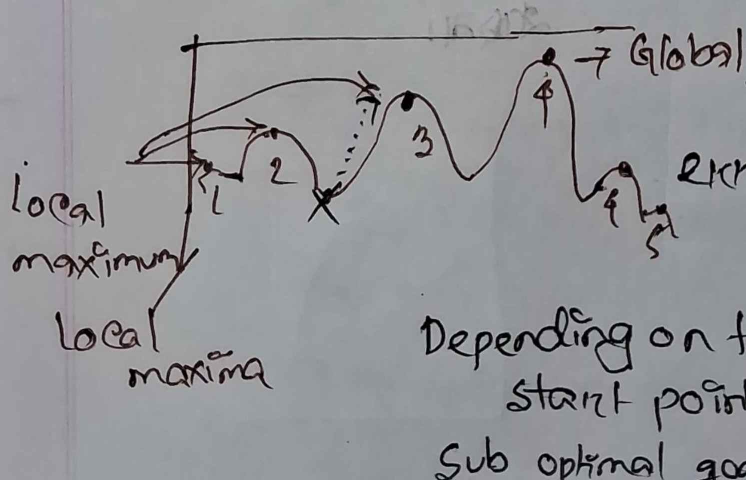
For simplicity আমরা $y = mx$ রাখি,

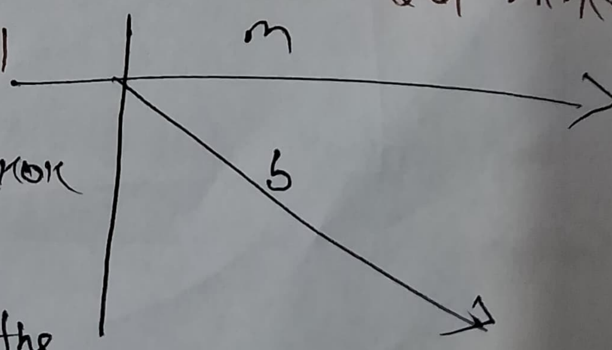
but ~~if~~ intersect exist করলে, $y = mx + b$

unknown unknown

দুইটা unknown axis এ লাগবে,

complex problem এর জন্য multiple top থাকতে পারে



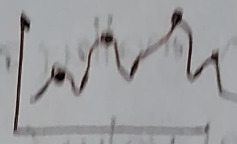


Depending on the start point, Sub optimal goal/top point return করে।

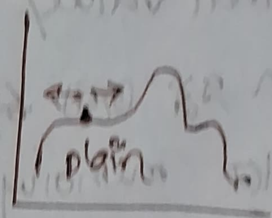
* n+1 unknown থাকলে figure হবে n+1 dimensional.

Disadvantage:

Sub optimal result ← ① Local maxima



② plateaus (stuck)

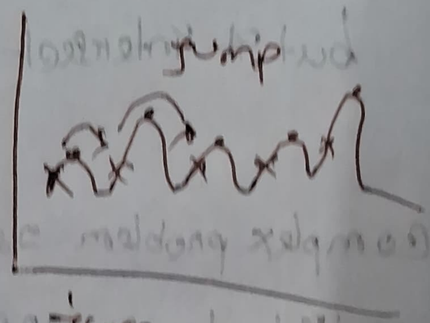


কোন direction খুঁজে পাওয়া যায় না। কোন দিকে গেলে right হবে।

③ Ridges

multiple local maxima একে-অপরের কাছাকাছি থাকে।

Iterative approach এ navigate করা খুব difficult হয়ে পড়ে।

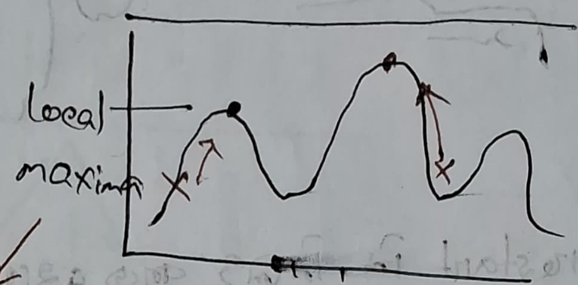


jump করেও approach করেনা।

Soln

Solutions:

① → Random start (Local maxima solve হয় কিন্তু plateaus, ridges নয়।)



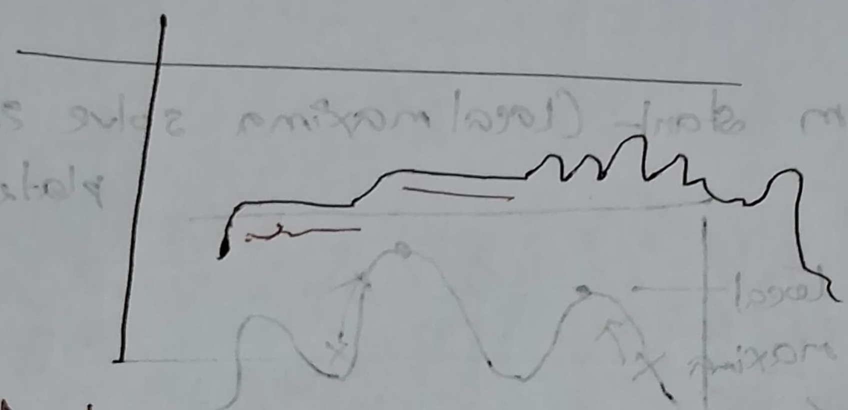
generally একটা top পর্যন্ত যায় terminate হয়ে যায়, কিন্তু এখন hill এ diff. point এ আবার restart দিলে, তারপর iterate করে আরেকটা maxima পেলাম, এভাবে বারবার start pos change করে, we'll try to find global maxima

how many time to restart → যদি 4-5 টা maxima থাকে আমরা দেখা যাবে 12-20 বার point restart দিলে পেয়ে যাবে

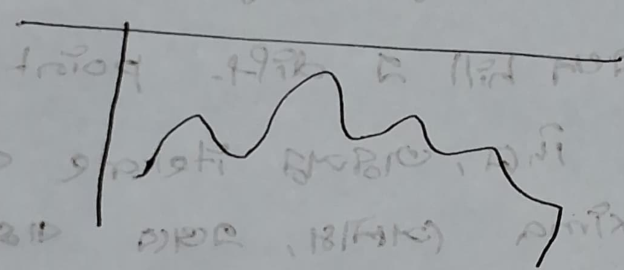
→ goal সম্পর্কে idea থাকলে, যেক্ষেত্রে goal এর কাছাকাছি percentage পেলে আর restart করবো না

→ maximum iteration value set করে দেওয়া যায়

② problem reformulation

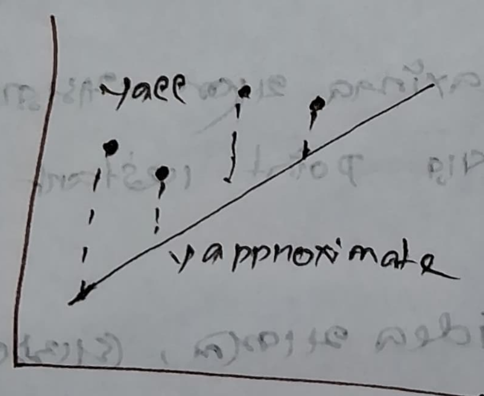


randomly restart দিয়ে নিলেও কাজ হবে না,



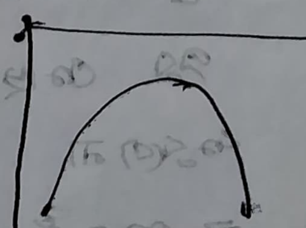
hill এর shape change করে।

(2)



$$\sum (y_{acc} - y_{approximate})^2$$

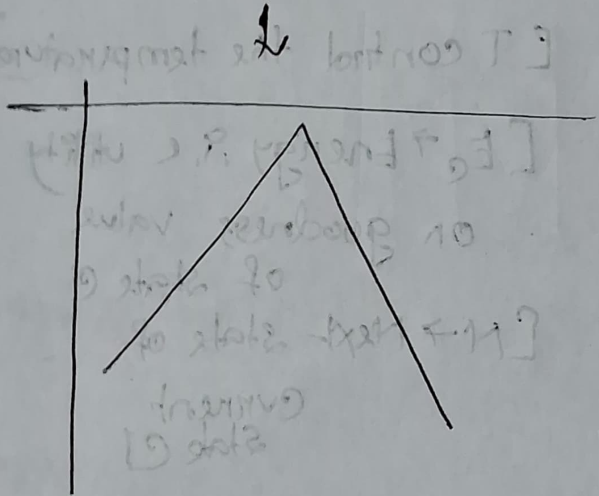
$$y = x^2$$



ⓐ ~~$y = x^2$~~

ⓑ $\sum |y_{acc} - y_{app}|$

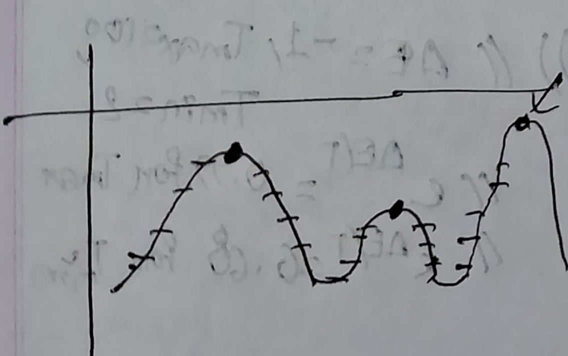
$y = |x|$



③ Simulated annealing

Initially একটা temperature value থাকবে,

→ high to low হতে থাকবে,



temperature value যত low হবে descend করার capacity তত কমতে থাকবে পরে একপর্যায়ে terminate করে, তখন top point থেকে সে যেতে- পারে because temperature low.

Simulated annealing

$C = C_{init}$

$T = T_{max}$ to T_{min} [T control the temperature]

$E_C = E(C)$ [$E_C \rightarrow$ Energy i.e utility or goodness value of state C

$N = Next(C)$ [$N \rightarrow$ Next state of current state C]

$E_N = E(N)$ [$E_N \rightarrow$ Energ i.e utiliy of state N]

$\Delta E = E_N - E_C$

if $(\Delta E > 0)$

$C = N$

Else if $(e^{\Delta E/T} > rand(0,1))$ // $\Delta E = -1$, $T_{max} = 100$, $T_{min} = 2$

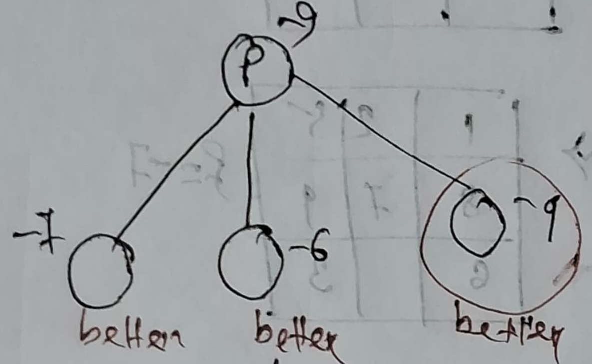
$C = N$ // $e^{\Delta E/T} = 0.99$ for T_{max}

// $e^{\Delta E/T} = 0.68$ for T_{min}

END

Hill climbing approaches

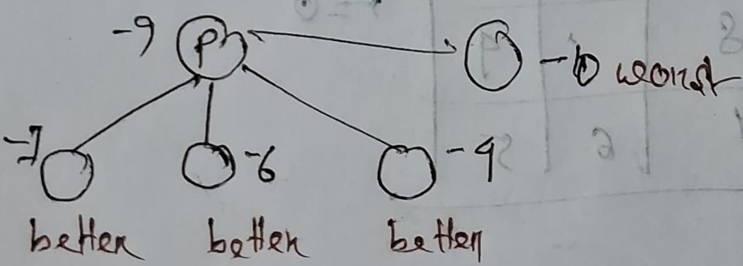
→ Steepest Hill climb



ধরে নিব step যেদিক থেকে সবথেকে তাড়াতাড়ি goal এ পৌঁছানো যাবে।

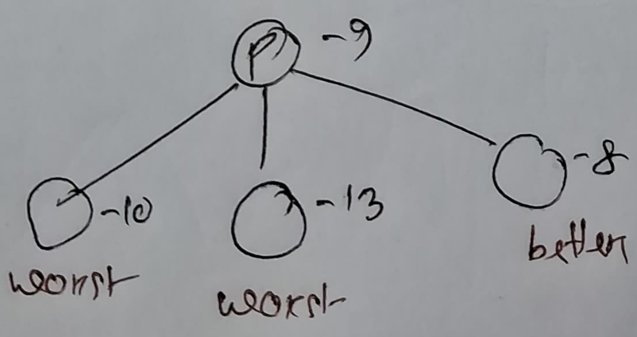
সবচেয়ে very best steepest edge count করবে

→ Stochastic Hill climb



better গুলো থেকে randomly যে কোনো একটা নিবে

→ First choice



First যে children parent এর তুলনায় better হয় তাকেই নেয়.

Pros: numerous children থাকলে, তখন time save করে, কারণ সব আলাদা check করা লাগে না।

Local Optimum

Plateaus

Start

1	2	5
8	7	4
	6	3

$f = -6$

1	2	5
	7	4
8	6	3

$f = -7$

1	2	5
8	7	4
6		3

$f = -7$

$f = -$ manhattan distance

goal

1	2	3
8		4
7	6	5

$f = 0$